```
import requests
import time
import re
import logging
import os

# Logging setup
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S"
)
log = logging.getLogger(__name__)

# Config
BOT_TOKEN   = os.environ.get("BOT_TOKEN", "")
CHANNEL_ID  = os.environ.get("CHANNEL_ID", "")
CHANNEL_TAG = "@BlockAlerts"

ACCOUNTS = [
    "zachxbt",
    "PeckShieldAlert",
    "CertiKAlert",
    "lookonchain",
    "realScamSniffer",
]

NITTER_INSTANCES = [
    "https://nitter.privacydev.net",
    "https://nitter.poast.org",
    "https://nitter.cz",
    "https://nitter.1d4.us",
]

KEYWORDS = [
    "hack", "exploit", "drain", "drained", "stolen", "steal",
    "breach", "attack", "scam", "rug", "phish", "phishing",
    "freeze", "frozen", "blacklist", "launder", "laundering",
    "bounty", "alert", "warning", "suspicious", "compromise",
    "vulnerability", "social engineering", "just in", "breaking",
    "transfer", "moved", "whale", "million", "billion",
]

POLL_INTERVAL   = 90
```

```
BETWEEN_USERS   = 5
REQUEST_TIMEOUT = 15

seen_ids = set()


def get_nitter_rss(username):
    for base in NITTER_INSTANCES:
        url = base + "/" + username + "/rss"
        try:
            r = requests.get(url, timeout=REQUEST_TIMEOUT, headers={
                "User-Agent": "Mozilla/5.0 (compatible; NewsBot/1.0)"
            })
            if r.status_code == 200 and "<rss" in r.text:
                log.info("  Got RSS for @" + username + " via " + base)
                return r.text
            else:
                log.warning("  " + base + " returned " + str(r.status_code) + " f
        except Exception as e:
            log.warning("  " + base + " failed: " + str(e))
    log.error("  All Nitter instances failed for @" + username)
    return None


def parse_rss(xml, username):
    tweets = []
    items = re.findall(r"<item>(.*?)</item>", xml, re.DOTALL)
    for item in items:
        guid = re.search(r"<guid[^>]*>(.*?)</guid>", item)
        if not guid:
            continue
        tweet_id = guid.group(1).strip()

        desc = re.search(r"<description>(.*?)</description>", item, re.DOTALL)
        if not desc:
            continue
        raw = desc.group(1)
        raw = re.sub(r"<!\[CDATA\[|\]\]>", "", raw)
        raw = re.sub(r"<[^>]+>", " ", raw)
        raw = re.sub(r"&amp;", "&", raw)
        raw = re.sub(r"&lt;", "<", raw)
        raw = re.sub(r"&gt;", ">", raw)
        raw = re.sub(r"&quot;", '"', raw)
        raw = re.sub(r"&#39;", "'", raw)
        text = re.sub(r"\s+", " ", raw).strip()

        link = re.search(r"<link>(.*?)</link>", item)
```

```
            url = link.group(1).strip() if link else ""
            for base in NITTER_INSTANCES:
                url = url.replace(base, "https://twitter.com")

            tweets.append({"id": tweet_id, "text": text, "url": url, "username": user
        return tweets


def is_relevant(tweet):
    text = tweet["text"].lower()
    username = tweet["username"].lower()

    if username in ("zachxbt", "realscamsniffer"):
        return True

    return any(kw in text for kw in KEYWORDS)


def extract_amount(text):
    m = re.search(r"\$[\d,.]+\s*[MBKmb]?", text)
    return m.group(0).strip() if m else ""


def source_emoji(username):
    mapping = {
        "zachxbt":          "🕵️",
        "peckshieldalert": "🛡️",
        "certikalert":     "🔐",
        "lookonchain":     "🐋",
        "realscamsniffer": "🚨",
    }
    return mapping.get(username.lower(), "📡")


def format_message(tweet):
    text     = tweet["text"]
    username = tweet["username"]
    url      = tweet["url"]
    amount   = extract_amount(text)
    emoji    = source_emoji(username)

    if amount:
        header = "⚠️ JUST IN: " + amount + " event detected"
    else:
        header = "⚠️ JUST IN: New Alert"

    body = re.sub(r"https?://\S+", "", text).strip()
```

```
    if len(body) > 600:
        body = body[:597] + "..."

    msg = (
        header + "\n\n" +
        body + "\n\n" +
        emoji + " Source: @" + username + "\n" +
        "🔗 " + url + "\n\n" +
        CHANNEL_TAG
    )
    return msg


def send_telegram(text):
    url = "https://api.telegram.org/bot" + BOT_TOKEN + "/sendMessage"
    payload = {
        "chat_id": CHANNEL_ID,
        "text": text,
        "parse_mode": "HTML",
        "disable_web_page_preview": False,
    }
    try:
        r = requests.post(url, json=payload, timeout=REQUEST_TIMEOUT)
        if r.status_code == 200:
            return True
        else:
            log.error("Telegram error " + str(r.status_code) + ": " + r.text)
            return False
    except Exception as e:
        log.error("Telegram send failed: " + str(e))
        return False


def run_cycle():
    log.info("Starting new cycle...")
    new_posts = 0

    for username in ACCOUNTS:
        log.info("Fetching @" + username + "...")
        xml = get_nitter_rss(username)

        if not xml:
            time.sleep(BETWEEN_USERS)
            continue

        tweets = parse_rss(xml, username)
        log.info("  Found " + str(len(tweets)) + " tweets")
```

```
        for tweet in tweets:
            if tweet["id"] in seen_ids:
                continue

            seen_ids.add(tweet["id"])

            if not is_relevant(tweet):
                log.info("  Skipped (not relevant): " + tweet["text"][:60])
                continue

            msg = format_message(tweet)
            success = send_telegram(msg)

            if success:
                new_posts += 1
                log.info("  Posted: " + tweet["text"][:60])
            else:
                log.warning("  Failed to post: " + tweet["text"][:60])

            time.sleep(2)

        time.sleep(BETWEEN_USERS)

    log.info("Cycle done. " + str(new_posts) + " new alerts posted.")


def main():
    log.info("BlockAlerts bot starting...")
    log.info("Tracking: " + ", ".join(ACCOUNTS))
    log.info("Posting to: " + CHANNEL_ID)
    log.info("Poll interval: " + str(POLL_INTERVAL) + "s")

    log.info("Seeding seen tweet IDs...")
    for username in ACCOUNTS:
        xml = get_nitter_rss(username)
        if xml:
            tweets = parse_rss(xml, username)
            for t in tweets:
                seen_ids.add(t["id"])
        time.sleep(BETWEEN_USERS)
    log.info("Seeded " + str(len(seen_ids)) + " existing IDs. Only NEW tweets wil

    while True:
        try:
            run_cycle()
        except Exception as e:
```

```
            log.error("Cycle crashed: " + str(e))
        log.info("Sleeping " + str(POLL_INTERVAL) + "s...")
        time.sleep(POLL_INTERVAL)


if __name__ == "__main__":
    main()
```